DIAGRAMA DE CLASES UML

Scooby-Doo — Misterio de Cartagena

Arquitectura orientada a objetos para C++ y Qt

Modelo de clases, herencia, composición, asociaciones y flujo de ejecución

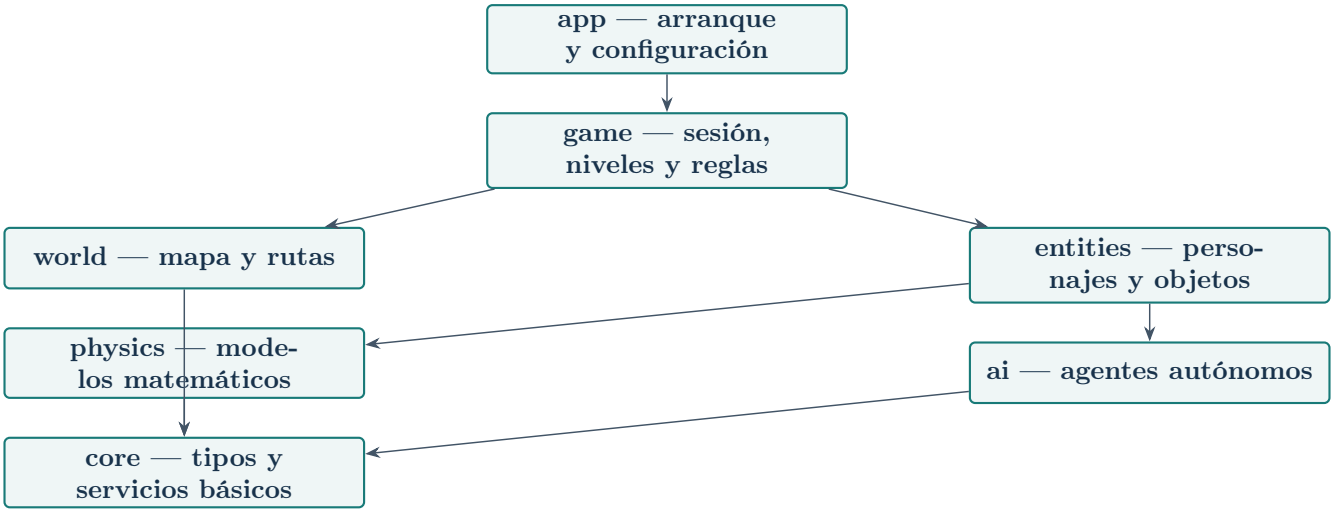
Diseño modular distribuido por paquetes para mantener legible el diagrama completo.

Convenciones UML utilizadas

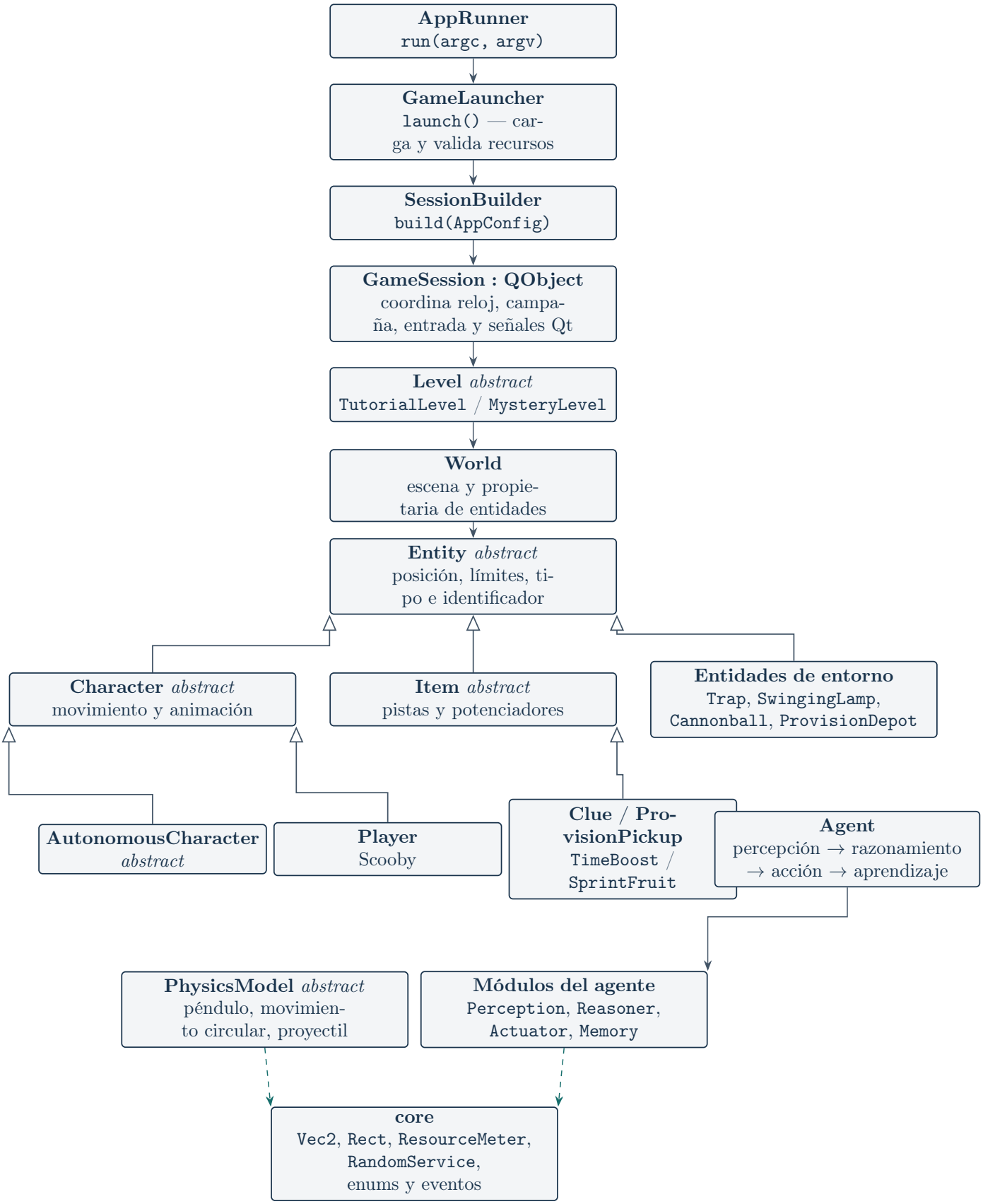
+ público	Operación o atributo accesible desde otras clases.
- privado	Detalle interno de implementación.
# protegido	Miembro accesible en la clase y sus derivadas.
Línea continua + triángulo vacío	Generalización / herencia; el triángulo apunta a la clase base.
Rombo negro	Composición; el extremo con rombo es el propietario del ciclo de vida.
Línea continua con flecha	Asociación dirigida o conocimiento de otra clase; no implica propiedad.
Línea discontinua con flecha	Dependencia de uso temporal.
Multiplicidad	`1`, `0..1`, `0..*`, `1..*` indican cuántas instancias participan.

Criterio de presentación. El modelo tiene muchas clases y responsabilidades; por eso se organiza en láminas verticales por paquete y una vista global. No se amontonan todas las clases en una única página ilegible. La notación se mantiene consistente en todo el documento.

Paquetes del sistema

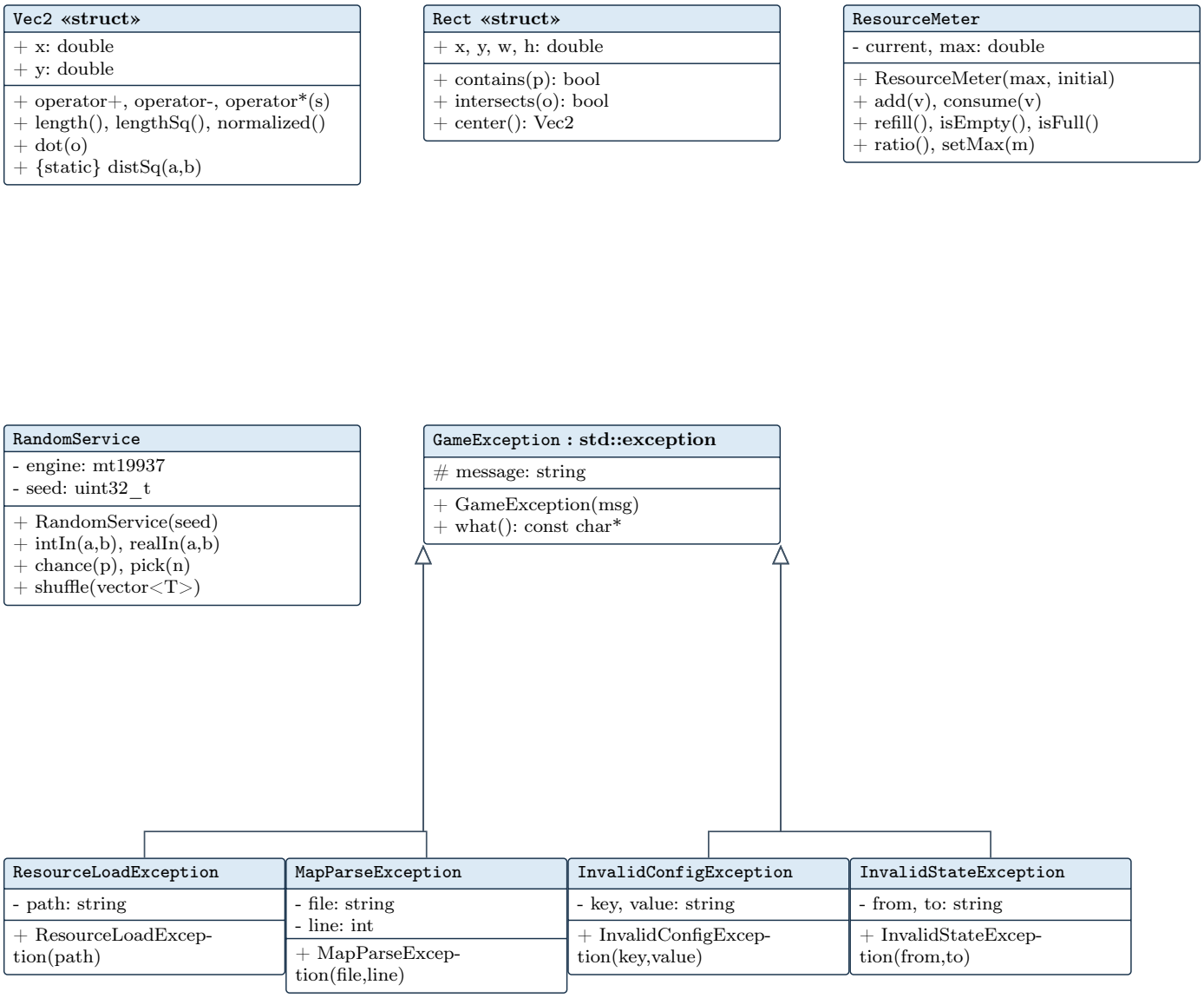


1. Vista global de clases y flujo vertical



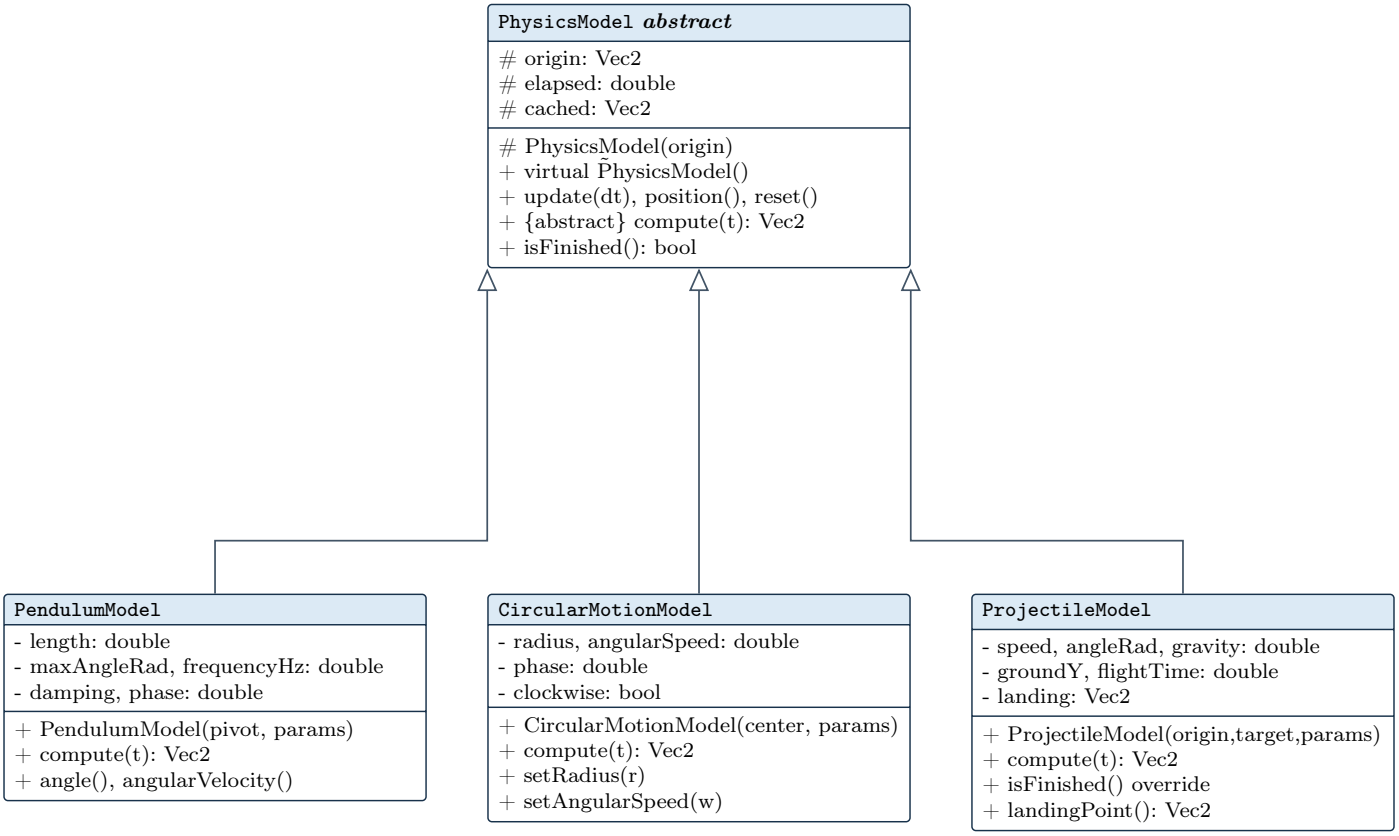
La vista global resume el flujo de control y las familias principales; las siguientes páginas detallan atributos, métodos y relaciones.

2. Paquete `core`: tipos, recursos, aleatoriedad y excepciones



Responsabilidad del paquete. Los tipos geométricos y servicios básicos no dependen de Qt. `RandomService` centraliza la aleatoriedad; `ResourceMeter` reutiliza la misma lógica para tiempo, provisiones y estamina. Las excepciones se propagan hasta el arranque, donde se gestionan con salida controlada.

3. Paquete `physics`: modelos matemáticos

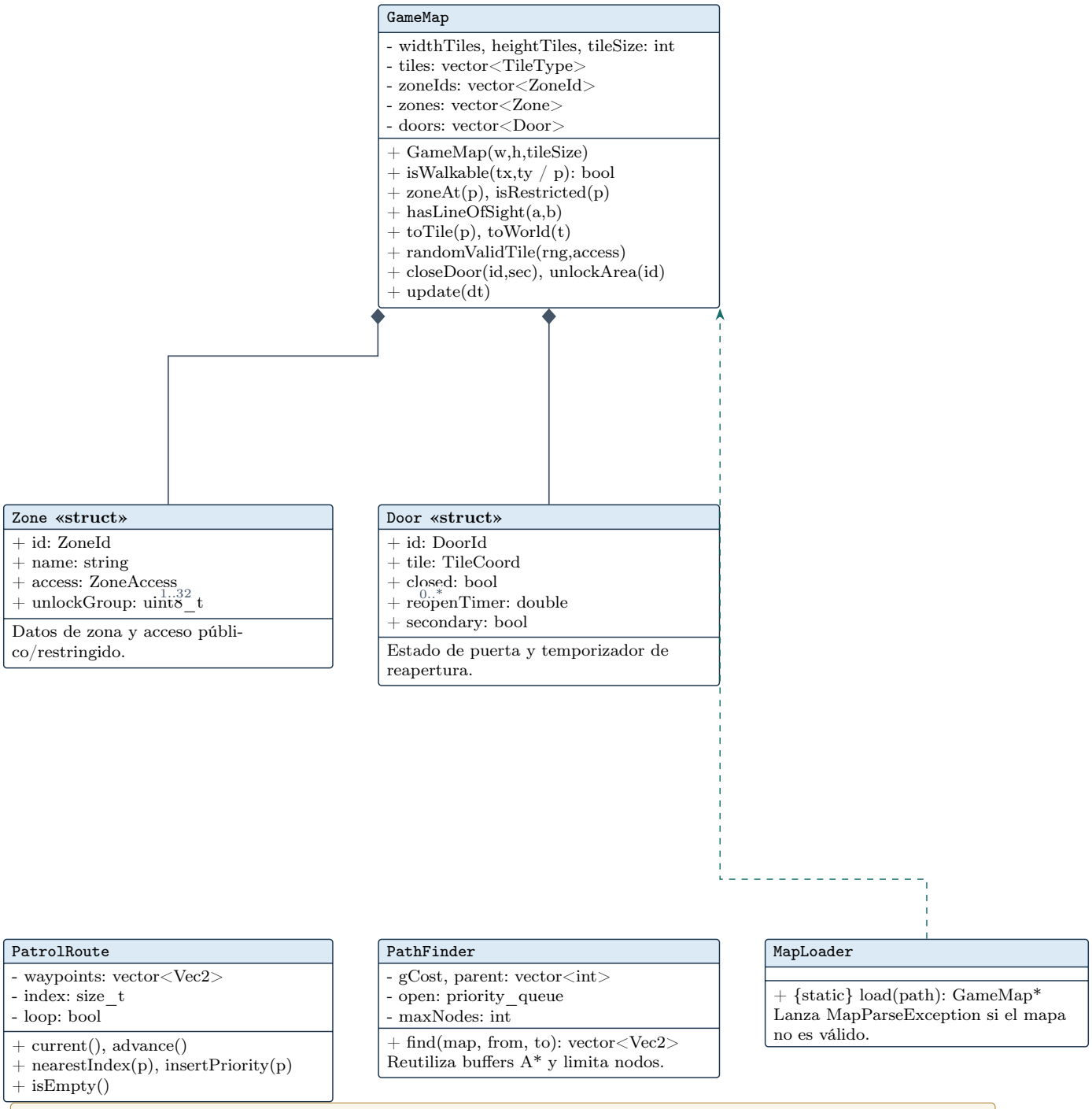


Uso previsto en el juego

- `PendulumModel`: lámpara colgante que oscila en arco.
- `CircularMotionModel`: vigilante que patrulla en órbita alrededor del depósito.
- `ProjectileModel`: trayectoria parabólica de un proyectil en el segundo nivel; es la tercera física especial propuesta en el diseño.

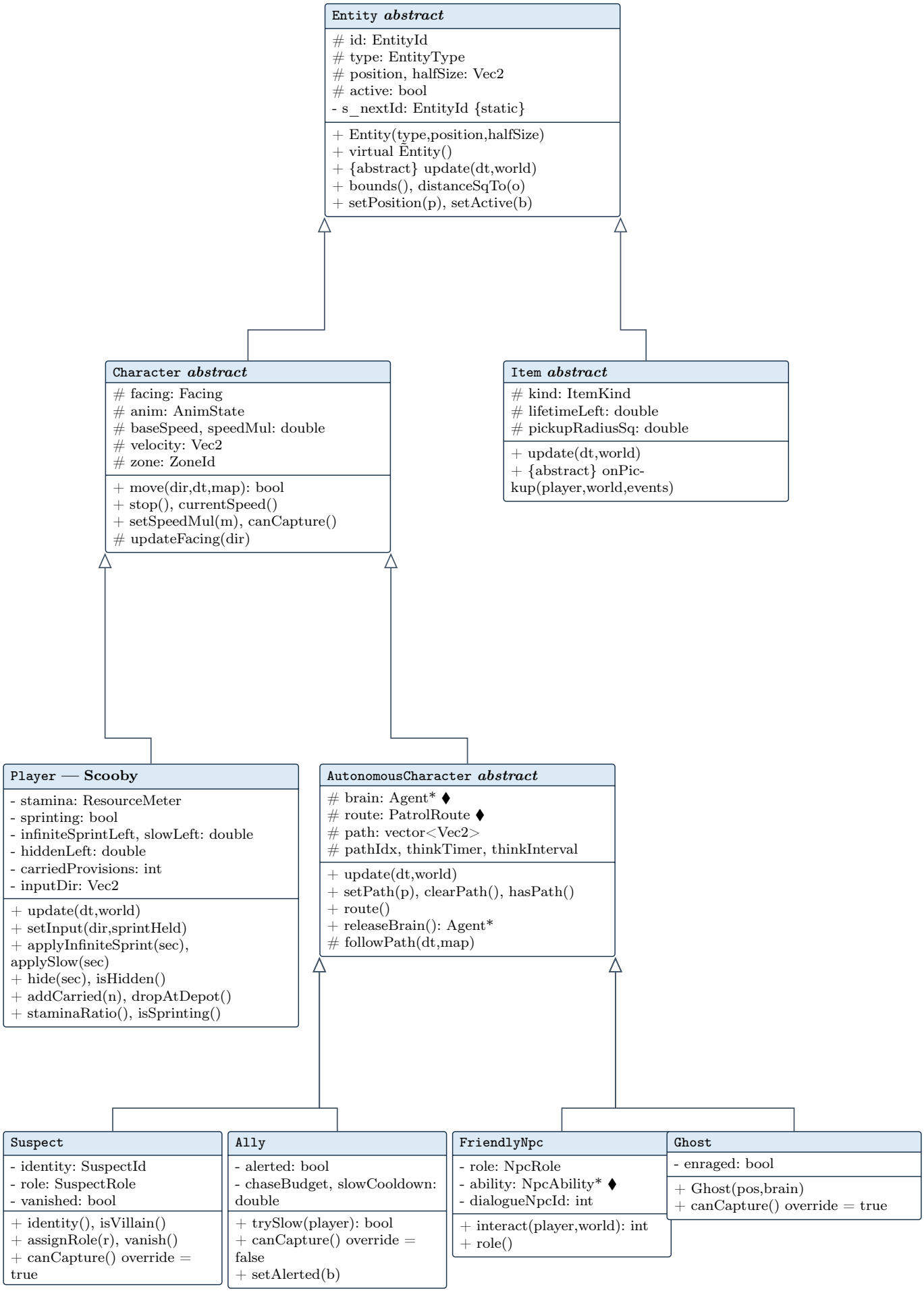
Los parámetros `PendulumParams`, `CircularParams` y `ProjectileParams` los produce `DifficultyProfile`.

4. Paquete `world`: mapa, zonas, puertas y navegación

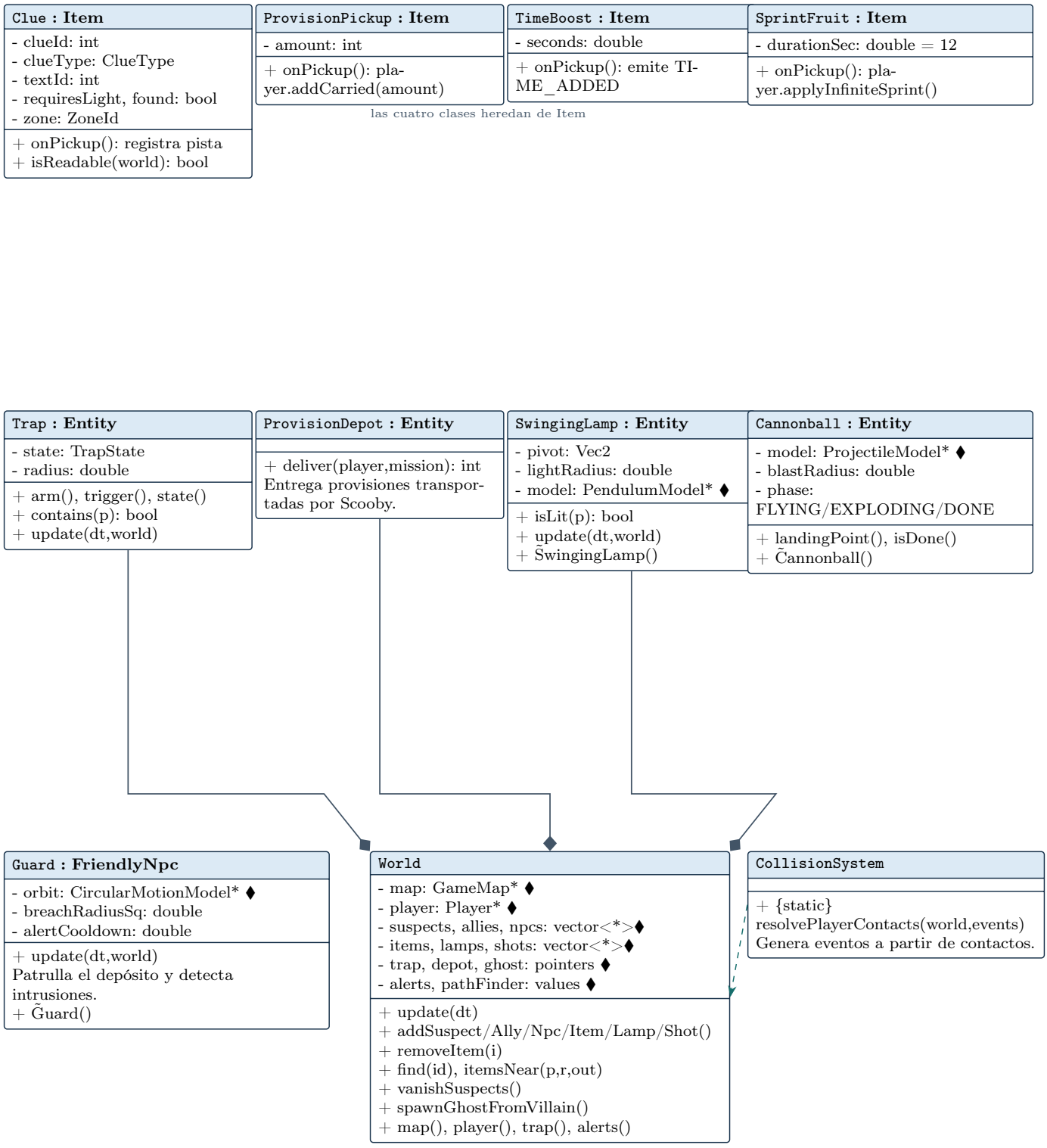


Reglas de navegación. El mapa almacena tiles en un vector 1D (índice `y * width + x`); calcula línea de visión con Bresenham; las puertas cerradas bloquean visión y tránsito. `PathFinder` se instancia una sola vez dentro de `World` y reutiliza memoria entre búsquedas.

5. Paquete `entities`: jerarquía de entidades y personajes

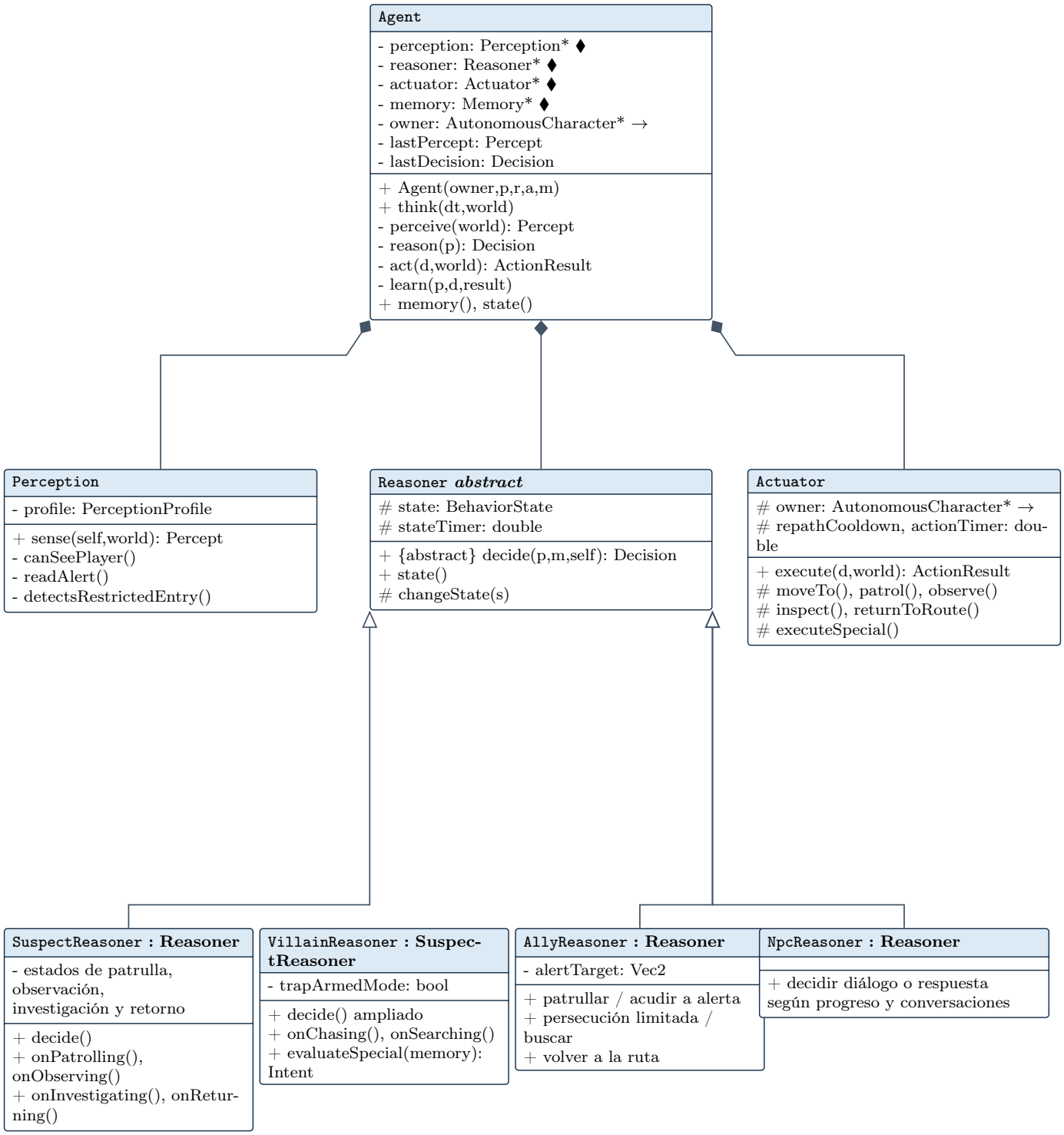


6. Entidades concretas, objetos y propiedad de la escena



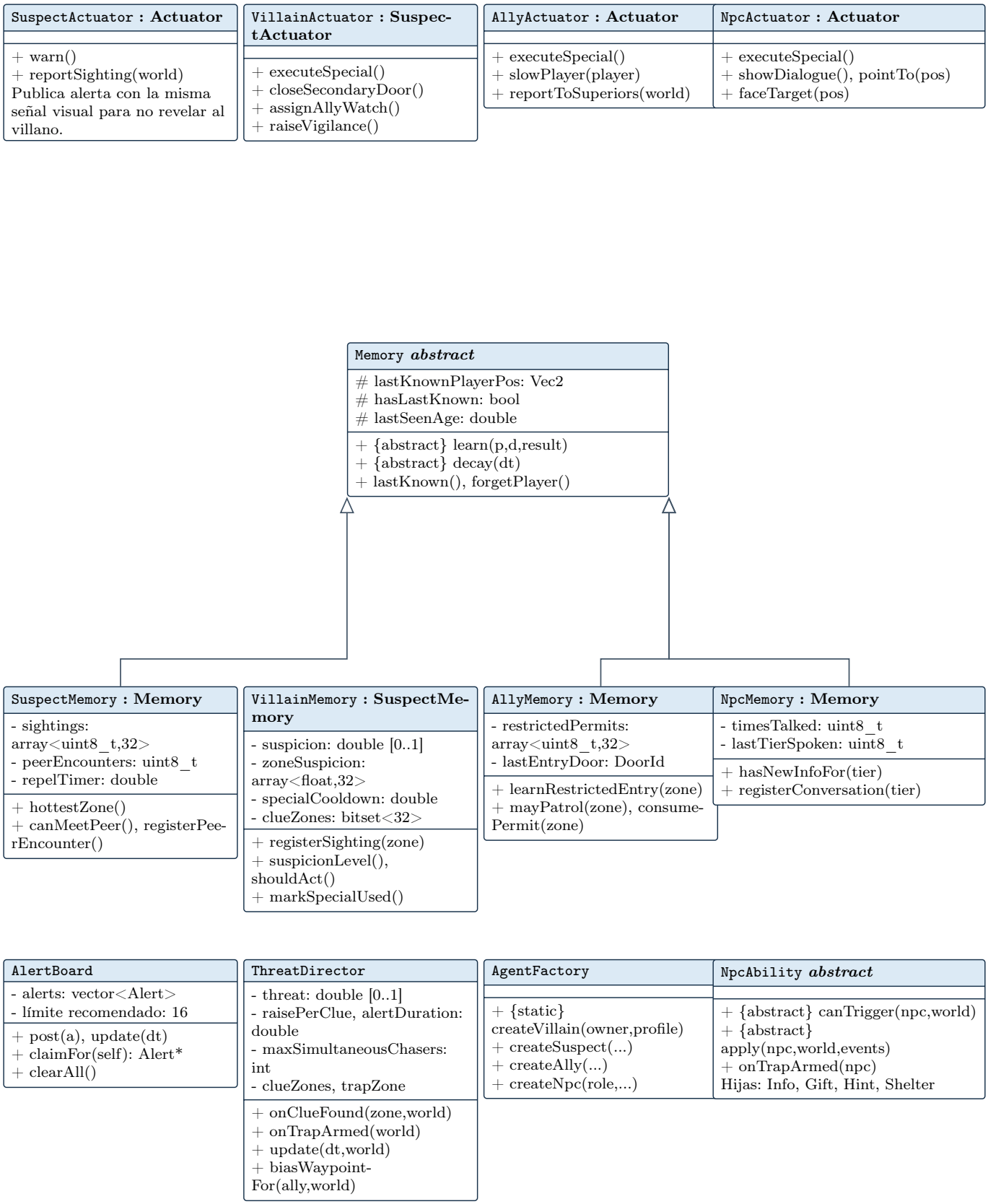
La composición indica propiedad del ciclo de vida; las colecciones de `World` son responsables de destruir cada entidad exactamente una vez.

7. Paquete `ai`: ciclo del agente y razonamiento

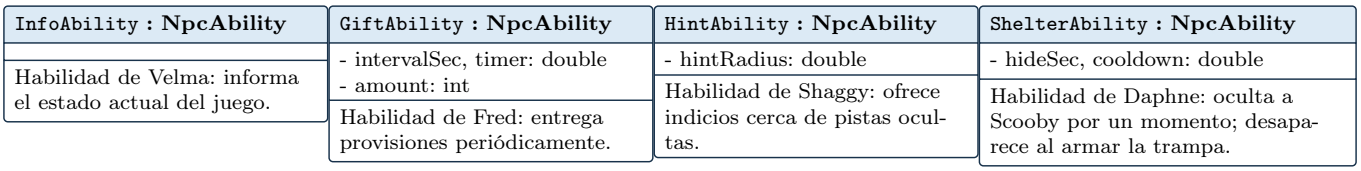


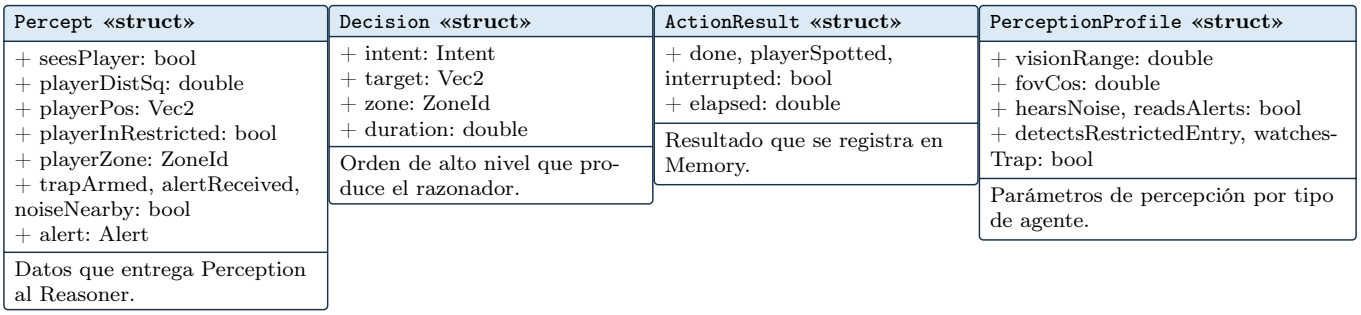
Ciclo de decisión obligatorio. `Agent::think()` ejecuta, en orden, percepción → razonamiento → acción → aprendizaje. El módulo `Perception` produce un `Percept`; el razonador genera una `Decision`; el actuador devuelve `ActionResult`; la memoria registra el resultado y modifica decisiones futuras.

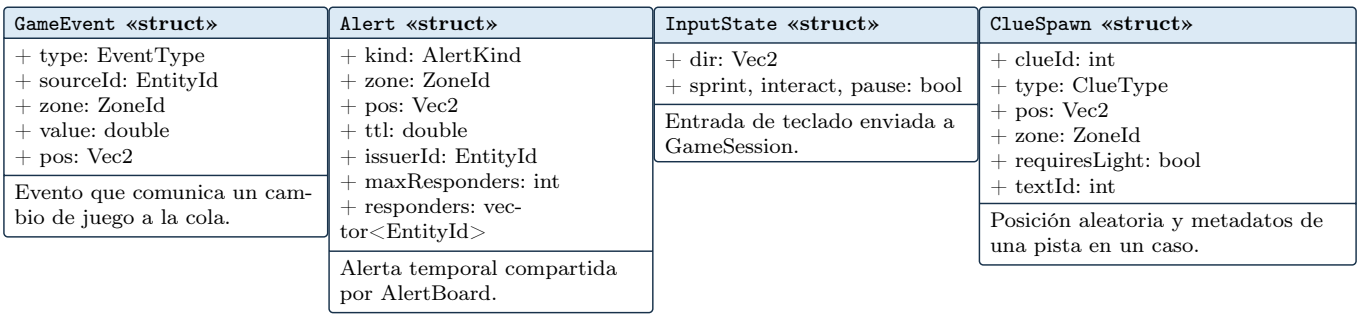
8. Paquete `ai`: actuadores, aprendizaje y habilidades NPC



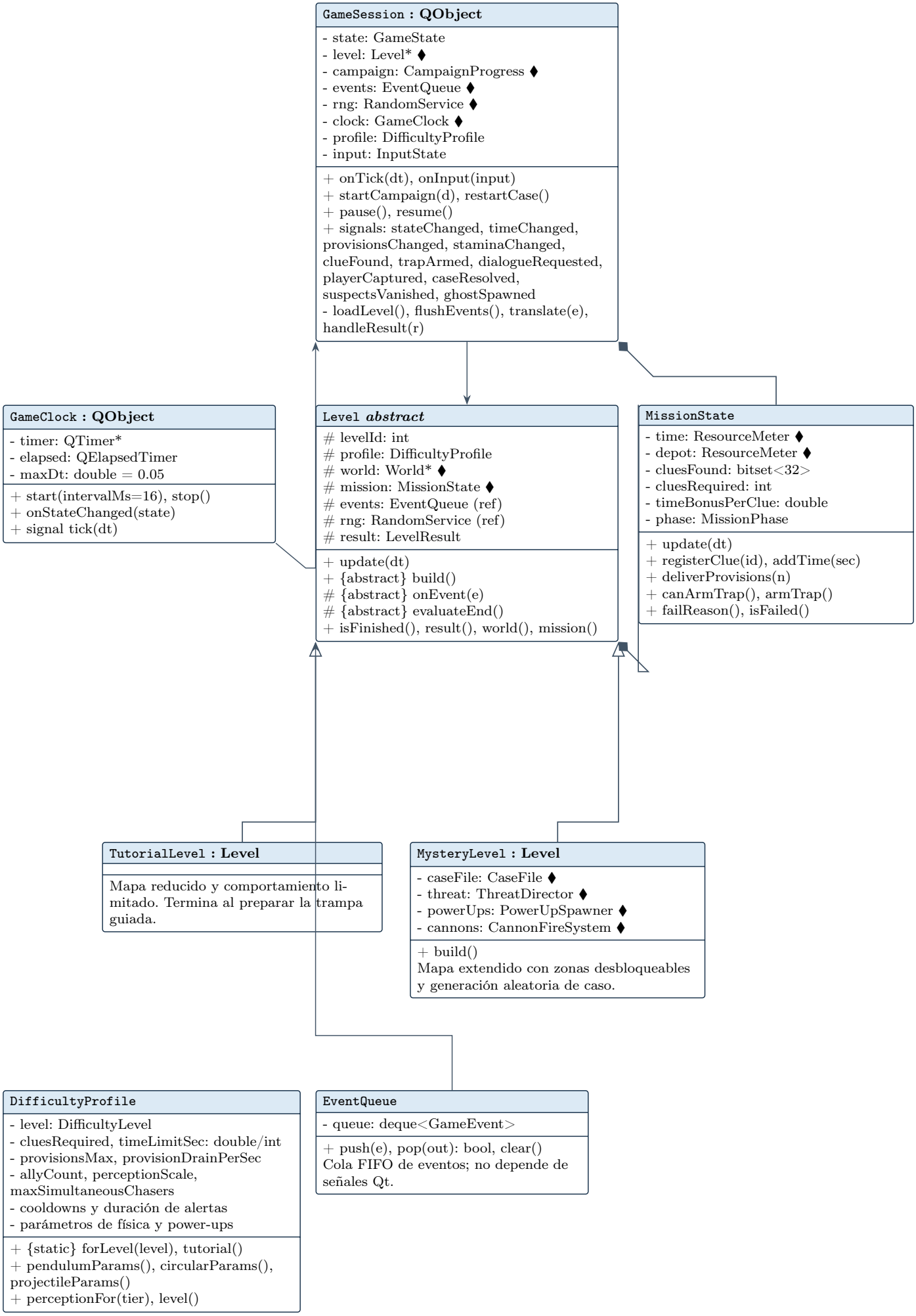
9. Habilidades de NPC y tipos de datos del agente



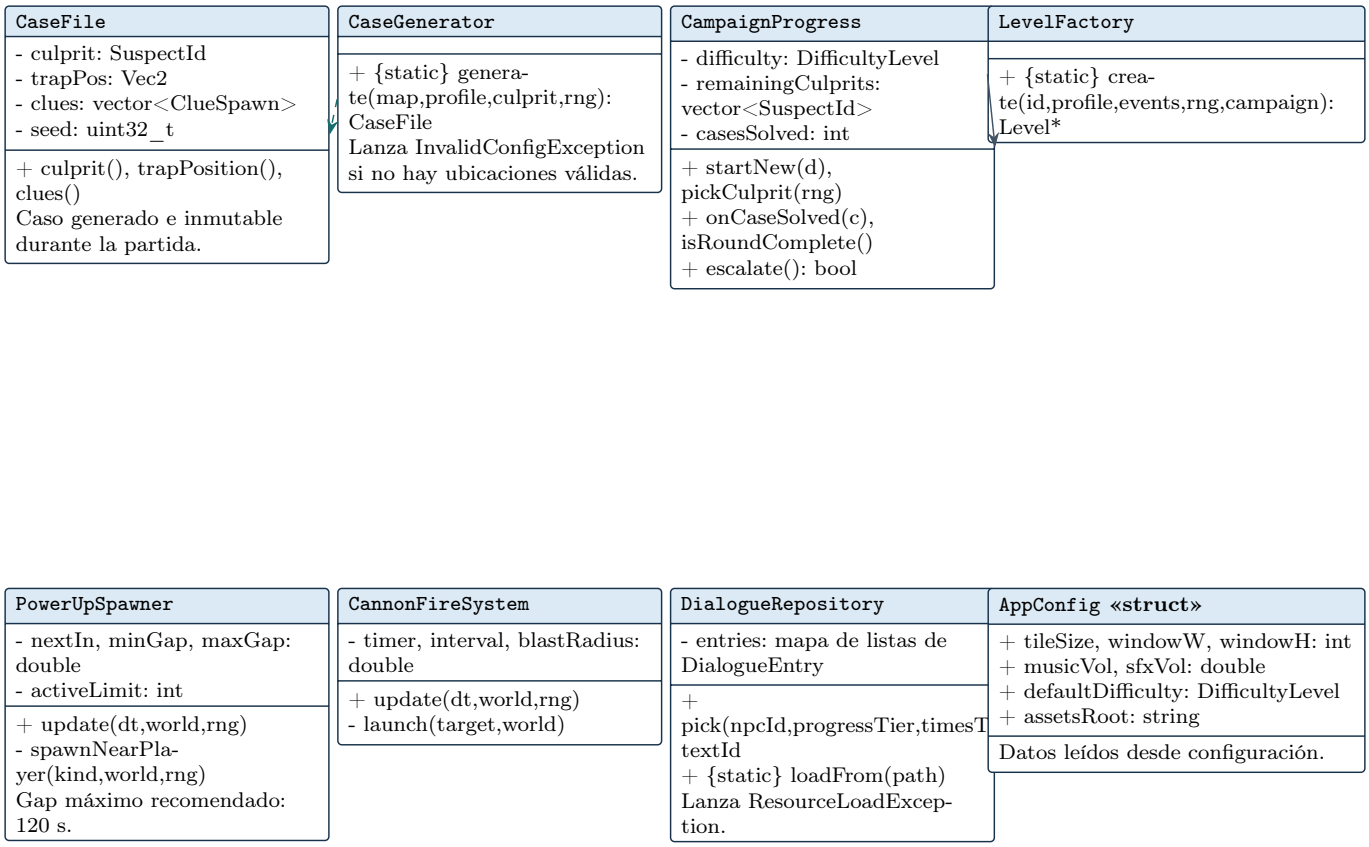




10. Paquete `game`: sesión, niveles, misión y dificultad

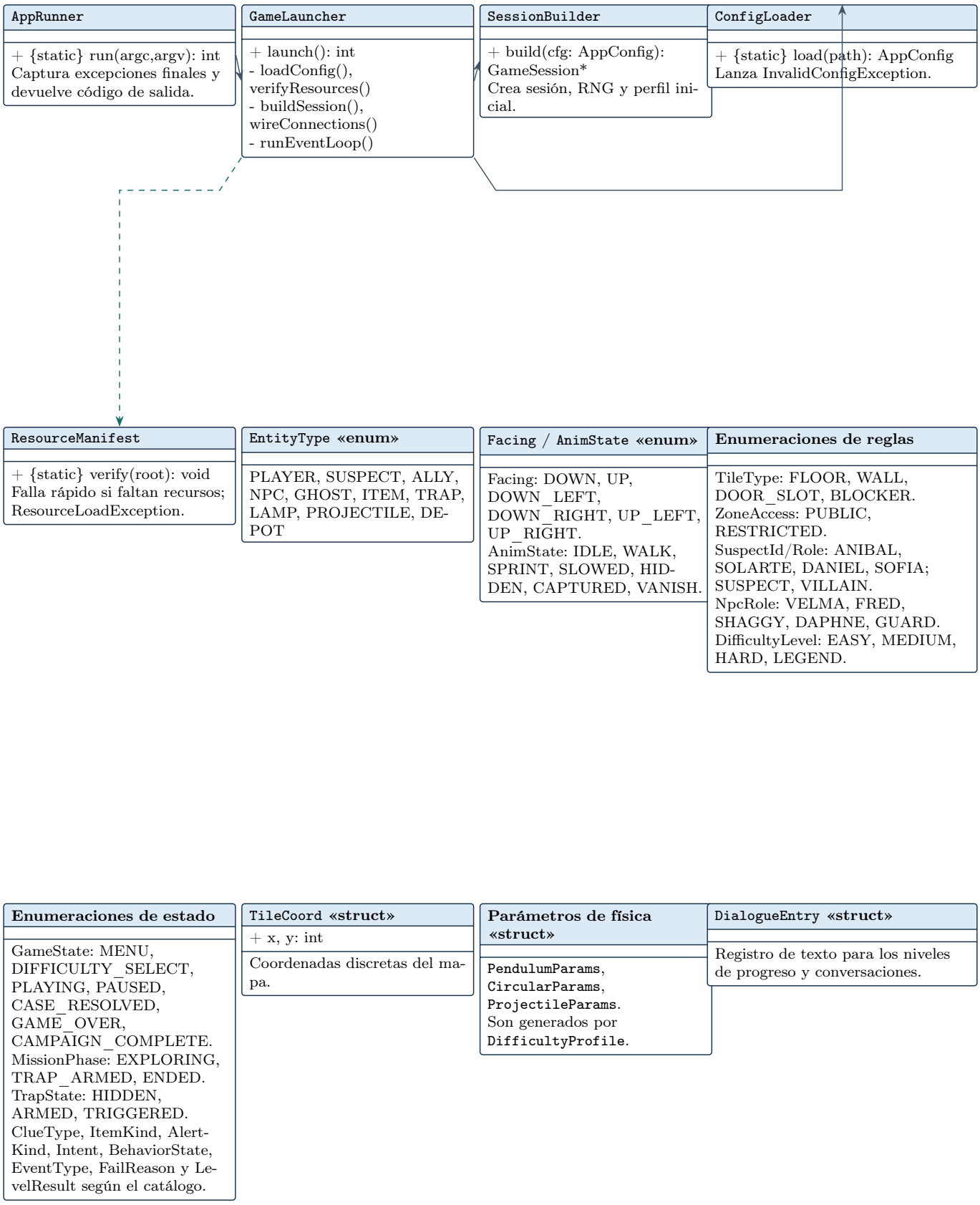

GameSession : QObject
- state: GameState
- level: Level* ♦
- campaign: CampaignProgress ♦
- events: EventQueue ♦
- rng: RandomService ♦
- clock: GameClock ♦
- profile: DifficultyProfile
- input: InputState
+ onTick(dt), onInput(input)
+ startCampaign(d), restartCase()
+ pause(), resume()
+ signals: stateChanged, timeChanged, provisionsChanged, staminaChanged, clueFound, trapArmed, dialogueRequested, playerCaptured, caseResolved, suspectsVanished, ghostSpawned
- loadLevel(), flushEvents(), translate(e), handleResult(r)
GameClock : QObject
- timer: QTimer*
- elapsed: QElapsedTimer
- maxDt: double = 0.05
+ start(intervalMs=16), stop()
+ onStateChanged(state)
+ signal tick(dt)
Level abstract
levelId: int
profile: DifficultyProfile
world: World* ♦
mission: MissionState ♦
events: EventQueue (ref)
rng: RandomService (ref)
result: LevelResult
+ update(dt)
+ {abstract} build()
{abstract} onEvent(e)
{abstract} evaluateEnd()
+ isFinished(), result(), world(), mission()
MissionState
- time: ResourceMeter ♦
- depot: ResourceMeter ♦
- cluesFound: bitset<32>
- cluesRequired: int
- timeBonusPerClue: double
- phase: MissionPhase
+ update(dt)
+ registerClue(id), addTime(sec)
+ deliverProvisions(n)
+ canArmTrap(), armTrap()
+ failReason(), isFailed()
TutorialLevel : Level
Mapa reducido y comportamiento limitado. Termina al preparar la trampa guiada.
MysteryLevel : Level
- caseFile: CaseFile ♦
- threat: ThreatDirector ♦
- powerUps: PowerUpSpawner ♦
- cannons: CannonFireSystem ♦
+ build()
Mapa extendido con zonas desbloqueables y generación aleatoria de caso.
DifficultyProfile
- level: DifficultyLevel
- cluesRequired, timeLimitSec: double/int
- provisionsMax, provisionDrainPerSec
- allyCount, perceptionScale, maxSimultaneousChasers
- cooldowns y duración de alertas
- parámetros de física y power-ups
+ {static} forLevel(level), tutorial()
+ pendulumParams(), circularParams(), projectileParams()
+ perceptionFor(tier), level()
EventQueue
- queue: deque<GameEvent>
+ push(e), pop(out): bool, clear()
Cola FIFO de eventos; no depende de señales Qt.

11. Paquete game: caso, campaña y sistemas secundarios



Responsabilidades. `CaseGenerator` selecciona culpable, posiciones de pistas y lugar de activación de la trampa. `CampaignProgress` mantiene el ciclo de sospechosos y escala la dificultad. `PowerUpSpawner` limita la frecuencia de aparición de potenciadores; `CannonFireSystem` administra la física parabólica opcional; `DialogueRepository` centraliza los textos y evita repetir información sin progreso.

12. Paquete app, enumeraciones y tipos compartidos



13. Tipos de datos y catálogos de enumeraciones

Enumeraciones previstas en el modelo

`EntityType`	PLAYER, SUSPECT, ALLY, NPC, GHOST, ITEM, TRAP, LAMP, PROJECTILE, DEPOT.
`Facing`	DOWN, UP, DOWN_LEFT, DOWN_RIGHT, UP_LEFT, UP_RIGHT. Las seis orientaciones vinculan la dirección de movimiento con las filas del sprite sheet.
`AnimState`	IDLE, WALK, SPRINT, SLOWED, HIDDEN, CAPTURED, VANISH.
`TileType` / `ZoneAccess`	FLOOR, WALL, DOOR_SLOT, BLOCKER / PUBLIC, RESTRICTED.
`SuspectId` / `SuspectRole`	ANIBAL, SOLARTE, DANIEL, SOFIA / SUSPECT, VILLAIN.
`NpcRole` / `DifficultyLevel`	VELMA, FRED, SHAGGY, DAPHNE, GUARD / EASY, MEDIUM, HARD, LEGEND.
`GameState`	MENU, DIFFICULTY_SELECT, PLAYING, PAUSED, CASE_RESOLVED, GAME_OVER, CAMPAIGN_COMPLETE.
`MissionPhase` / `TrapState`	EXPLORING, TRAP_ARMED, ENDED / HIDDEN, ARMED, TRIGGERED.
`ClueType` / `ItemKind`	IDENTITY, TRAP_LOCATION, HINT / CLUE, PROVISION, TIME_BOOST, SPRINT_FRUIT.
`AlertKind`	ZONE_WATCH, SIGHTING_REPORT, RESTRICTED_BREACH, CLUE_FOUND, TRAP_ARMED.
`Intent`	IDLE, PATROL, OBSERVE, APPROACH, INVESTIGATE, CHASE, SEARCH, GO_TO_ALERT, RETURN_TO_ROUTE, WARN, REPORT, SLOW_PLAYER, TALK, POINT_TO, FACE_PLAYER, VANISH, HUNT, SPECIAL_CLOSE_DOOR, SPECIAL_ASSIGN_WATCH, SPECIAL_RAISE_VIGILANCE.
`BehaviorState`	PATROLLING, OBSERVING, APPROACHING, INVESTIGATING, CHASING, SEARCHING, ALERTED, RETURNING, HUNTING, VANISHED, TALKING.
`EventType`	CLUE_FOUND, ITEM_PICKED, PROVISIONS_DELIVERED, TIME_ADDED, TRAP_ARMED, TRAP_TRIGGERED, PLAYER_CAPTURED, PLAYER_SLOWED, PLAYER_HIDDEN, ALERT_RAISED, DOOR_CLOSED, DOOR_OPENED, SUSPECTS_VANISHED, GHOST_SPAWNED, DIALOGUE_STARTED.
`FailReason` / `LevelResult`	CAUGHT, OWN_TRAP, TIME_UP, PROVISIONS_EMPTY / RUNNING, TUTORIAL_DONE, CASE_WON, CASE_LOST.

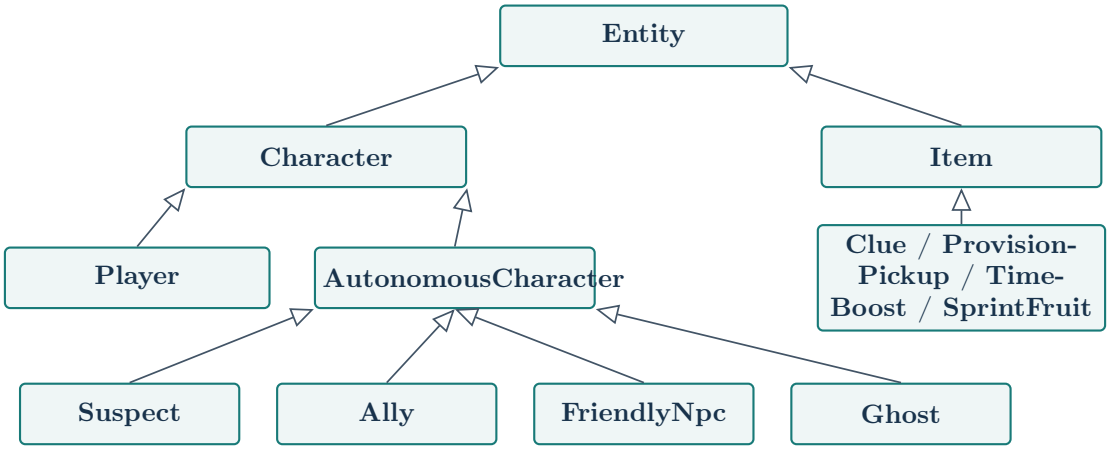
Estructuras de datos sin lógica compleja

`GameEvent`	`type`, `sourceId`, `zone`, `value`, `pos`.
`Alert`	`kind`, `zone`, `pos`, `ttl`, `issuerId`, `maxResponders`, `responders`.
`Percept`	Detección del jugador, distancia al cuadrado, posición, entrada restringida, zona, trampa activada, alerta y ruido.
`Decision`	`intent`, `target`, `zone`, `duration`.
`ActionResult`	`done`, `playerSpotted`, `interrupted`, `elapsed`.
`PerceptionProfile`	Alcance visual, cono de visión, escucha, lectura de alertas, detección de zona restringida y trampa.
`InputState`	`dir`, `sprint`, `interact`, `pause`.
`ClueSpawn`	Identificador, tipo, posición, zona, requisito de luz e identificador del texto de la pista.
`AppConfig`	Tamaño de tile, resolución de ventana, volúmenes, dificultad inicial y raíz de recursos.

14. Resumen de relaciones: herencia y generalización

Herencia principal

`Entity`	`Character`, `Item`, `Trap`, `SwingingLamp`, `Cannonball`, `ProvisionDepot`.
`Character`	`Player`, `AutonomousCharacter`.
`AutonomousCharacter`	`Suspect`, `Ally`, `FriendlyNpc`, `Ghost`.
`FriendlyNpc`	`Guard`.
`Item`	`Clue`, `ProvisionPickup`, `TimeBoost`, `SprintFruit`.
`PhysicsModel`	`PendulumModel`, `CircularMotionModel`, `ProjectileModel`.
`Reasoner`	`SuspectReasoner` (base de `VillainReasoner`), `AllyReasoner`, `NpcReasoner`.
`Actuator`	`SuspectActuator` (base de `VillainActuator`), `AllyActuator`, `NpcActuator`.
`Memory`	`SuspectMemory` (base de `VillainMemory`), `AllyMemory`, `NpcMemory`.
`NpcAbility`	`InfoAbility`, `GiftAbility`, `HintAbility`, `ShelterAbility`.
`Level`	`TutorialLevel`, `MysteryLevel`.
`GameException`	`ResourceLoadException`, `MapParseException`, `InvalidConfigException`, `InvalidStateException`.
`QObject`	`GameClock`, `GameSession`.



15. Composición, multiplicidad y propiedad de memoria

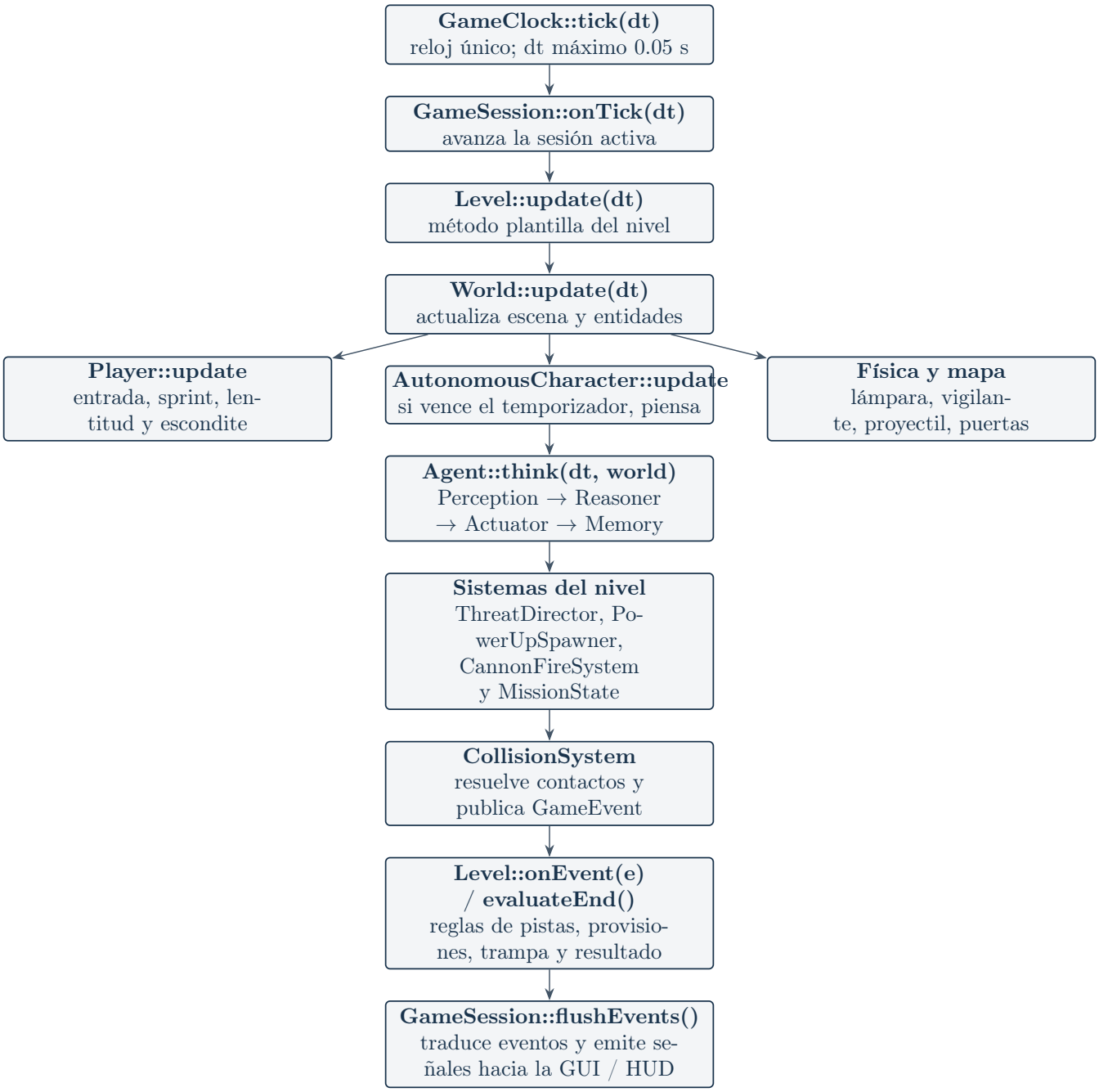
Composición y asociaciones principales

Propietario	Mult.	Elemento contenido / referencia
`GameSession`	$1 \to 0..1$	Posee `Level`; también posee `CampaignProgress`, `EventQueue`, `RandomService` y `GameClock`.
`Level`	$1 \to 1$	Posee `World` y `MissionState`; referencia `EventQueue` y `RandomService`.
`MysteryLevel`	$1 \to 1$	Posee `CaseFile`, `ThreatDirector`, `PowerUpSpawner` y `CannonFireSystem`.
`World`	$1 \to 1$	Posee `GameMap`, `Player`, `Trap`, `ProvisionDepot`, `AlertBoard` y `PathFinder`.
`World`	$1 \to 4$	Posee los cuatro sospechosos definidos en el caso.
`World`	$1 \to 0..*$	Posee aliados, ítems, lámparas y proyectiles; posee NPCs (0..5) y fantasma (0..1).
`GameMap`	$1 \to 1..32$	Contiene zonas; además contiene 0..* puertas.
`AutonomousCharacter`	$1 \to 1$	Posee un `Agent` y una `PatrolRoute`.
`Agent`	$1 \to 1$ c/u	Posee `Perception`, `Reasoner`, `Actuator` y `Memory`.
`FriendlyNpc`	$1 \to 0..1$	Posee su `NpcAbility`.
`SwingingLamp / Guard / Cannonball`	$1 \to 1$	Poseen respectivamente `PendulumModel`, `CircularMotionModel` y `ProjectileModel`.
`Player`	$1 \to 1$	Posee `ResourceMeter` para la estamina.
`MissionState`	$1 \to 2$	Posee medidores de tiempo y provisiones mediante `ResourceMeter`.

Propiedad explícita. El camino de destrucción previsto es

`GameSession` → `Level` → `World` → mapa y entidades.
`AutonomousCharacter` destruye su `Agent`; `Agent` destruye sus cuatro módulos; `FriendlyNpc` destruye su habilidad.
`releaseBrain()` transfiere el agente del villano al fantasma y deja nulo el puntero original para evitar doble liberación.

16. Secuencia de ejecución: tick del juego y señales Qt



Conexiones Qt principales

- `GameClock::tick(double)` → `GameSession::onTick(double)`: avanzar simulación.
- `GameSession::stateChanged(GameState)` → `GameClock::onStateChanged`: pausar o reanudar el reloj.
- GUI → `GameSession::onInput(const InputState&)`: entrada del jugador.
- `timeChanged`, `provisionsChanged`, `staminaChanged`, `clueFound`: actualización del HUD.
- `playerCaptured(int)` y `caseResolved(bool)`: pantallas de resultado.

17. Notas de implementación y decisiones arquitectónicas

Reglas de diseño reflejadas en el diagrama

1. **Un solo reloj.** `GameClock` emite `tick(dt)`; las entidades no mantienen temporizadores Qt propios.
2. **Pensamiento desacoplado del movimiento.** Los agentes razonan a menor frecuencia, pero los personajes siguen moviéndose cada frame.
3. **Percepción progresiva.** Primero se filtra por distancia al cuadrado, después por campo de visión y finalmente por línea de visión.
4. **Propiedad explícita.** La composición indica quién destruye cada objeto; se evita asignar un mismo puntero propietario a dos contenedores.
5. **Sin singletons.** `RandomService` y `EventQueue` se inyectan por referencia donde corresponda.
6. **Dificultad como abstracción.** `DifficultyProfile` parametriza las reglas, la IA, los tiempos y las tres físicas.
7. **Villano discreto.** `VillainReasoner` amplía al razonador del sospechoso; la animación pública de aviso no revela su identidad.
8. **Transición a fantasma.** Al activar la trampa, se desvanecen los sospechosos y el agente del villano se transfiere al fantasma; los aliados intensifican la búsqueda.

Observaciones para la versión que se programará

- El documento incluye la tercera física propuesta: trayectoria parabólica de un proyectil para el Nivel 2. Debe conservarse solo si encaja con la rúbrica y con el tiempo de implementación.
- La interpretación adoptada es que los aliados pueden perseguir y ralentizar, pero no realizar la captura final; la captura la efectúa un sospechoso/villano o el fantasma, según la fase.
- La trampa usa la posición donde se activa como zona de captura. Si el interruptor y el punto de captura deben ser distintos, se deben añadir dos posiciones al modelo.
- La dirección de movimiento ofrece seis valores de orientación visual para sprites isométricos, aunque el control lógico se describa como cuatro direcciones principales más diagonales.

Fin del diagrama de clases. La arquitectura se ha separado por paquetes para facilitar su lectura, defensa y posterior implementación en C++/Qt.